



Figure 4: Signal slot mapping

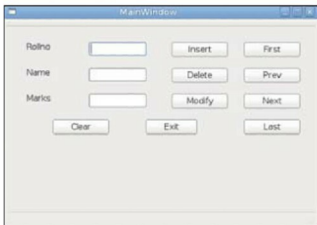


Figure 5: UI for simple database applications

From the project explorer, open *mainwindow.ui*, and place the following widgets on the form as shown in Figure 3:

- 3 Labels (object names—*lblNum1*, *lblNum2*, *lblNum3*)
- 3 Line Edits (object names—*leNum1*, *leNum2*, *leNum3*)
- 3 Push Buttons (object names—*pbAdd*, *pbClear*, *pbExit*)

To change object names of widgets, right click and select 'Change Object Name'. To change the display text of widgets, double click on them or right click and select 'Change Text'.

Enable the *readOnly* property for the third line edit with the object name *leNum3*. Use the property editor to change properties of a widget.

We now need to provide event handling for buttons. For our exit button, let's use the default signals and slot editor. Go to the signals and slot-mapping mode using the toolbar button (or F4 shortcut). Drag from the *Exit* button and drop to the empty area of the form, and a screen appears to configure the connection as shown in Figure 4. Select the check box at the bottom of the screen. Select the signal *clicked()* on the left pane and *close()* slot on the right pane. Make the mapping by clicking the *OK* button. Come back to the widget-editing mode using the toolbar button (or F3 shortcut).

When the project is built and run, you can close the application using the *Exit* button. But for the *Add* button the action is not available by default, so we need to create a custom slot for that.

Right click on the *Add* button—>Go to Slot. (Note that this feature is available only in the embedded form editor, and not when opened with the external designer.) Select the signal *clicked()* which will create a new function (slot) on *pbAdd_Clicked* in *mainwindow.cpp* and also its prototype in *mainwindow.h*. Put this code in that function's body:

```
int x,y;
x=QVariant(ui->leNum1->text()).toInt();
y=QVariant(ui->leNum2->text()).toInt();
z=x+y;
ui->leNum3->setText(QVariant(z).toString());
```

The *text()* function of *QLineEdit* class gives the text entered in the widget. Similarly, *setText(QString&)* is used to change the text inside. The *QVariant* class provides many constructors and functions for different types of conversion.

Similarly, you can use this code for a clear slot:

```
ui->leNum1->clear();
ui->leNum2->clear();
ui->leNum3->clear();
```

Alternatively, you can use the default signals and slot edit mode as clear slot for the line edit is predefined. Now build and run the project to test the application.

Database applications

Before going ahead with this example, let's prepare MySQL for connectivity. Create a database called *qtest*. Create a table called 'student' with the fields *rollno(int)*, *sname(varchar)*, *marks(decimal)* as follows:

```
mysql> create database qtest;
mysql> use qtest;
mysql> create table student(rollno int, sname varchar(10), marks
decimal(8,2) primary key(rollno));
```

Create a Qt GUI Project just like the previous example, but add the *QtSql* module this time.

You can observe the statement *QT += sql* in the project file. Design the form as shown in Figure 5. You can use the external designer for convenience. Right click on *UI Form*—>Open With—>Qt Designer.

Use the following object names for the widgets

- lblRollno*, *lblName*, *lblMarks* for labels
- leRollno*, *leName*, *leMarks* for line edits
- pbClear*, *pbExit*, *pbInsert*, *pbDelete*, *pbModify*, *pbFirst*, *pbPrev*, *pbNext*, *pbLast* for buttons.

You can use default slots for the *Clear* and *Exit* options.

Close Designer, come back to Qt Creator, and add these headers to *mainwindow.h*:

```
#include <QSqlDatabase>
#include <QSqlQuery>
```